

Table des matières

1. Description du projet

- 1.1. Objectif du projet
- 1.2. Analyse du besoin (Besoin & Solution)

2. Cahier des charges

- 2.1. Contexte et problématique
- 2.2. Public ciblé et cas d'utilisation
- 2.3. Spécifications fonctionnelles (Méthodologie 2CBP)
- 2.4. Spécifications non-fonctionnelles (Performance, Sécurité, Disponibilité)
- 2.5. Contraintes techniques et réglementaires (RGPD)

3. Architecture et Technologies

- 3.1. Stack Technologique (FastAPI, Vanilla JS, MariaDB)
- 3.2. Architecture logicielle (Design Patterns Factory & Strategy)
- 3.3. Infrastructure et Conteneurisation (Docker Compose)

4. Conception Technique

- 4.1. Design Interface (Zoning et Maquettes SPA)
- 4.2. Modélisation de la donnée (MCD/MLD)
- 4.3. Conception UML

- Diagramme de cas d'utilisation

- Diagramme de classes
- Diagramme de séquence (Flux d'un scan STS)

5. Mise en œuvre de la Sécurité

- 5.1. Authentification et Autorisation (JWT & Bcrypt)
- 5.2. Gestion des accès Cloud (AWS STS AssumeRole)
- 5.3. Protection de l'infrastructure (Nginx Reverse Proxy & Rate Limiting)

6. Qualité Logicielle et Tests

- 6.1. Stratégie de tests unitaires (Mocking via Moto)
- 6.2. Tests d'intégration et couverture
- 6.3. Pipeline CI/CD (GitHub Actions)

7. Manuel de déploiement

- 7.1. Configuration de l'environnement (.env)
- 7.2. Procédure d'installation (Setup script)

Conclusion et Perspectives

- Bilan du projet
- Évolutions futures (Multi-cloud, IA prédictive)

1. Description du projet

1.1. Objectif du projet

Le projet **CloudDiagnoze** a pour objectif de fournir une plateforme SaaS (Software as a Service) spécialisée dans l'audit, l'analyse et la surveillance automatisée des infrastructures Cloud, avec une focalisation initiale sur l'écosystème **Amazon Web Services (AWS)**. L'ambition est de permettre aux administrateurs système et aux Managed Service Providers (MSP) de transformer une infrastructure brute en une vision structurée et optimisée.



Le projet vise à automatiser la découverte des ressources critiques les services pivots : EC2 (calcul) et S3 (stockage).

Besoin

Dans un contexte de complexification croissante des environnements Cloud, les entreprises font face à trois défis majeurs que CloudDiagnoze cherche à résoudre :

- **Visibilité et Inventaire** : Les infrastructures modernes sont souvent fragmentées sur plusieurs régions. Il est difficile d'obtenir une vue d'ensemble instantanée des ressources actives et de leur configuration.
- **Maîtrise des Coûts (FinOps)** : L'absence de surveillance des ressources sous-utilisées ou mal dimensionnées (idle resources) entraîne un gaspillage financier significatif.
- **Sécurité et Conformité** : La détection manuelle des failles de sécurité, comme des buckets S3 ouverts au public ou des politiques d'accès trop permissives, est source d'erreurs critiques.
- **Externalisation** : Pour un prestataire de services (MSP), il existe un besoin crucial de démontrer visuellement et objectivement la valeur ajoutée d'une reprise en main de l'infrastructure client.

Solution

CloudDiagnoze répond à ces besoins par une solution logicielle entièrement conteneurisée s'appuyant sur la méthodologie **2CBP** (Cost, Compliance, Best Practices):

- **Analyse Automatisée sans Clés Statiques** : La solution utilise le protocole **AWS STS AssumeRole**. Cette approche garantit une sécurité maximale en utilisant des identifiants temporaires pour scanner les comptes clients sans exposer de clés d'accès à long terme.
- **Moteur de Scan Asynchrone** : Grâce à une architecture basée sur **FastAPI** et **BackgroundTasks**, l'utilisateur peut lancer des scans multi-régions en parallèle et suivre la progression en temps réel via des événements streaming (SSE).
- **Dashboard de Diagnostic** : Une interface moderne en **Vanilla JS** restitue les données sous forme de statistiques globales (CPU moyen, état des instances) et d'audits détaillés par service.
- **Historisation et Export** : Le système stocke chaque "snapshot" d'infrastructure dans une base de données **MariaDB**, permettant de suivre l'évolution de l'infrastructure dans le temps et d'exporter les rapports au format JSON pour des analyses externes

2. Cahier des charges

2.1. Contexte et objectif

Le projet s'inscrit dans une démarche de modernisation de l'audit Cloud pour les prestataires de services managés (MSP). L'objectif est de fournir un outil capable de dresser un état des lieux instantané et rigoureux d'une infrastructure AWS afin d'identifier les leviers d'optimisation avant toute reprise en gestion. Contrairement aux outils généralistes, **CloudDiagnoze** se concentre sur la précision technique des ressources critiques (calcul, stockage, réseau, base de données) pour générer des rapports de configuration exploitables en un seul endroit.

2.2. Analyse du besoin

Problématique

La gestion d'infrastructures Cloud multi-régions sans outil centralisé expose les organisations à plusieurs risques majeurs:

- **Angle mort sécuritaire** : Difficulté à détecter les ressources exposées (S3 publics, ports ouverts) ou les configurations non conformes aux bonnes pratiques AWS.
- **Dérive budgétaire** : Accumulation de ressources inutilisées ou surdimensionnées par manque de visibilité sur les métriques de performance réelles (CPU, IOPS, Network).
- **Complexité d'accès** : Les méthodes d'audit classiques reposent souvent sur des clés d'accès statiques risquées, là où une délégation de rôle temporaire est préconisée.

Public ciblé

Le projet s'adresse principalement à deux profils d'utilisateurs:

- **Administrateurs Cloud / DevOps** : Souhaitant automatiser leur inventaire technique et vérifier la posture de sécurité de leurs comptes.
- **Architectes Solution en MSP** : Utilisant l'outil comme levier d'avant-vente pour démontrer visuellement au client les optimisations possibles via un rapport d'audit.

Besoins exprimés

Pour répondre à ces problématiques, la solution doit couvrir les besoins suivants :

- **Découverte automatisée** : Identifier l'ensemble des instances EC2 et buckets S3 sur plusieurs régions AWS simultanément.
- **Audit de configuration** : Analyser les paramètres de chiffrement, de versioning, de cycle de vie et d'accès réseau pour chaque ressource.
- **Collecte de métriques** : Récupérer les données de performance historiques (via CloudWatch) pour justifier un redimensionnement (right-sizing).
- **Visualisation centralisée** : Offrir un tableau de bord unifié permettant de filtrer les ressources par région, par état ou par client.

Contraintes

Le développement doit respecter des contraintes strictes pour garantir la viabilité du projet:

- **Sécurité des données** : Utilisation impérative du protocole **STS AssumeRole** pour éviter le stockage de credentials AWS long-terme.
- **Performance** : Les scans doivent être asynchrones pour ne pas bloquer l'interface utilisateur lors de l'analyse de larges infrastructures.
- **Portabilité** : L'application doit être entièrement conteneurisée via **Docker** pour assurer un déploiement reproductible et isolé.
- **Réglementation (RGPD)** : Les données collectées (métadonnées techniques) doivent être isolées par utilisateur et stockées de manière sécurisée en base locale

2.3. Fonctionnalités

Exigences fonctionnelles

Le système doit permettre à l'utilisateur de réaliser les actions suivantes :

- **Gestion des comptes** : Inscription, connexion, et gestion du profil utilisateur incluant la configuration d'un rôle IAM AWS (Role ARN).
- **Lancement d'audits (Scans)** : Configurer un nouveau scan en sélectionnant le fournisseur (AWS), les services cibles (EC2, S3), et les régions spécifiques.
- **Suivi en temps réel** : Afficher la progression du scan en direct via un flux d'événements (Server-Sent Events).
- **Visualisation par Dashboards** : Accéder à des tableaux de bord interactifs (SPA) présentant les résultats globaux et détaillés par ressource (instances EC2 et buckets S3).
- **Historisation et Exportation** : Consulter l'historique des scans passés avec des filtres (dates, services), et exporter les résultats complets au format JSON.

Exigences non fonctionnelles

- **Sécurité stricte** : L'authentification au backend doit se faire par jeton JWT (HS256) stocké côté client. Les mots de passe doivent être hachés via `bcrypt`. L'accès aux infrastructures scannées ne doit utiliser que des identifiants temporaires via `STS AssumeRole`, interdisant l'usage de clés d'accès statiques.
- **Performance et asynchronisme** : Le moteur de scan doit être asynchrone (FastAPI Background Tasks) et paralléliser les requêtes multi-régions (via `asyncio.gather`) pour minimiser le temps d'attente.
- **Architecture Réactive** : L'interface doit être une Single Page Application (SPA) fluide, sans rechargement complet de la page.
- **Isolation des données** : Chaque utilisateur ne doit avoir accès qu'aux ressources et historiques de scans liés à son propre identifiant (`user_id`).

2.4. Technologies

Pour répondre aux exigences, la pile technologique (Stack) suivante a été sélectionnée :

- **Front-end** : HTML5, Vanilla JavaScript, et le framework CSS Tailwind CSS.
- **Back-end** : Python 3.11 avec le framework asynchrone FastAPI et Pydantic pour la validation des données. L'interaction avec AWS est gérée par le SDK Boto3.
- **Base de données** : MariaDB 11.2, interfacée via l'ORM SQLAlchemy 2.0.
- **Infrastructure et Déploiement** : Conteneurisation complète via Docker et Docker Compose (3 services : backend, base de données, et proxy Nginx). L'intégration continue (CI/CD) est assurée par un pipeline GitHub Actions déclenchant automatiquement lint, tests et build Docker sur chaque PR vers master.

2.5. Critères de validation

Le projet sera considéré comme abouti si les critères suivants sont validés :

- **Validation des tests** : Succès de l'ensemble des tests unitaires et d'intégration (utilisant la librairie `moto` pour simuler l'environnement AWS sans coûts).
- **Intégrité de l'infrastructure** : Le script de déploiement (`docker-compose up`) lève les trois conteneurs (MariaDB, Backend, Nginx) de manière saine et fonctionnelle.
- **Audit fonctionnel** : Un utilisateur peut s'inscrire, configurer un Role ARN valide, lancer un scan, et visualiser les ressources récupérées sur le dashboard sans erreur serveur.

2.6. Charte graphique

L'interface utilisateur a été pensée pour refléter l'aspect technique et moderne des outils DevOps et Cloud.

- **Design System** : Approche "mobile-first" et responsive, avec un thème sombre (Dark mode) par défaut pour réduire la fatigue visuelle.
- **Palette de couleurs** :
 - *Couleur primaire* : Bleu (#137FEC) 
 - *Couleur secondaire* : Violet (#8B5CF6) 
 - *Couleur d'accentuation* : Cyan (#06B6D4) 
 - *Couleur de succès* : Vert (#10B981) 
- **Typographie** : Utilisation de *Rajdhani* pour le texte principal (aspect tech), *Orbitron* pour les titres et *JetBrains Mono* pour l'affichage de code ou de données brutes.
- **Animations et UX** : Intégration d'effets visuels dynamiques tels que des mots-clés flottants sur la page de connexion, des effets "scan line", des flux de particules de données et des effets de brillance (pulse/glow) sur les éléments interactifs pour renforcer l'immersion

3. Organisation

Pour mener à bien ce projet de Bachelor, une méthodologie de travail itérative et agile a été adoptée, inspirée du framework Scrum. Cette approche permet de s'adapter aux défis techniques rencontrés tout au long du développement.

- **Gestion de projet et Sprints** : Le développement est découpé en itérations (sprints) couvrant l'analyse, le développement du backend (moteur de scan), l'intégration frontend, et les tests.
 - **Versionning et Collaboration** : Utilisation de Git avec un dépôt centralisé (ex: GitHub/GitLab) pour le suivi du code source.
 - **Automatisation (CI/CD)** : Intégration d'un pipeline GitHub Actions (.github/workflows/ci.yml) automatisant l'exécution des tests (lint Ruff, tests unitaires via Moto, tests d'intégration avec MariaDB éphémère) ainsi que le build des images Docker (Backend et Frontend) à chaque merge sur master. modification majeure du code.
-

4. Conception

Cette section détaille l'architecture visuelle et logicielle de l'application CloudDiagnoze, illustrant la séparation stricte entre l'interface utilisateur (Front-end) et la logique métier (Back-end).

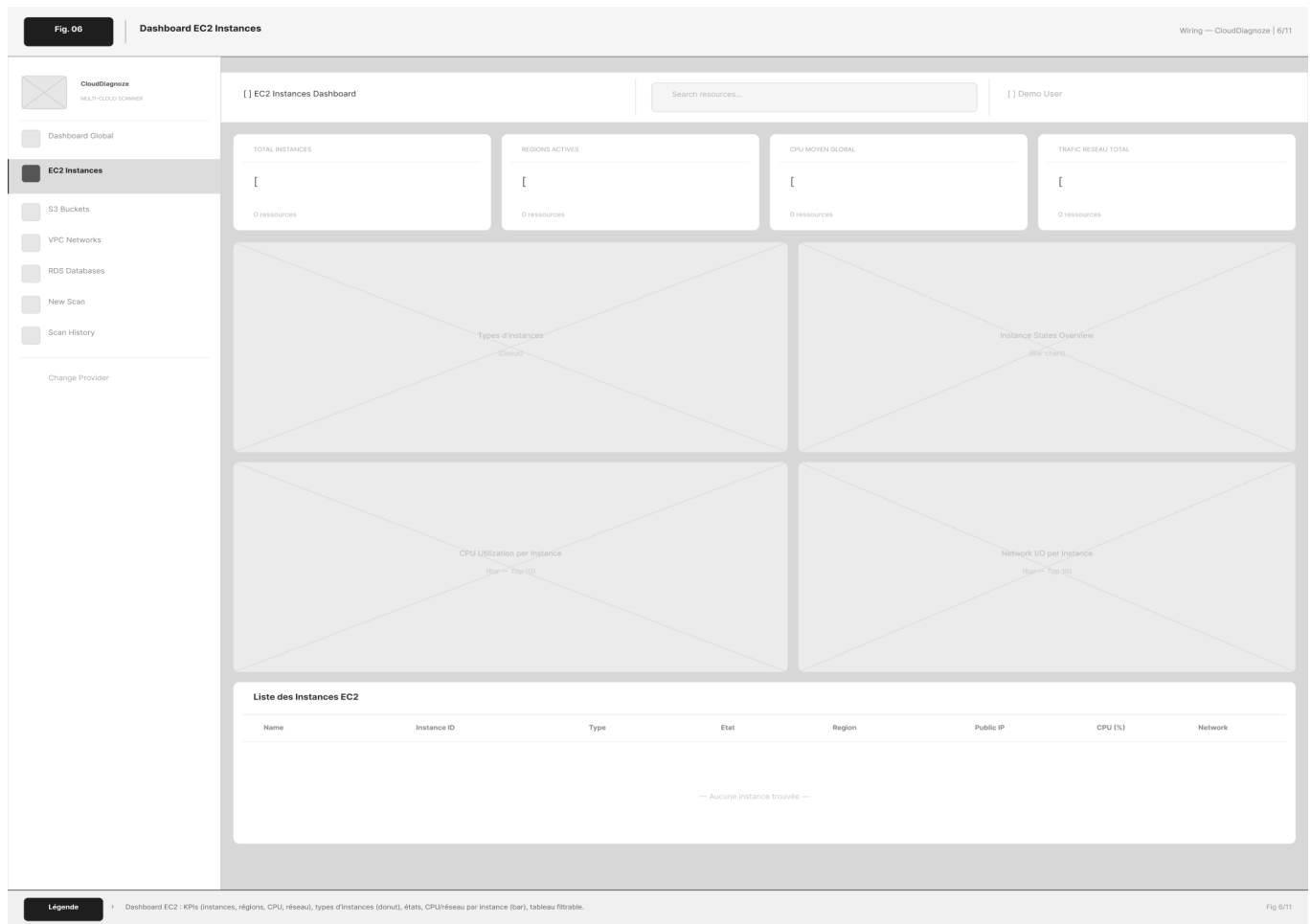
4.1. Front-end

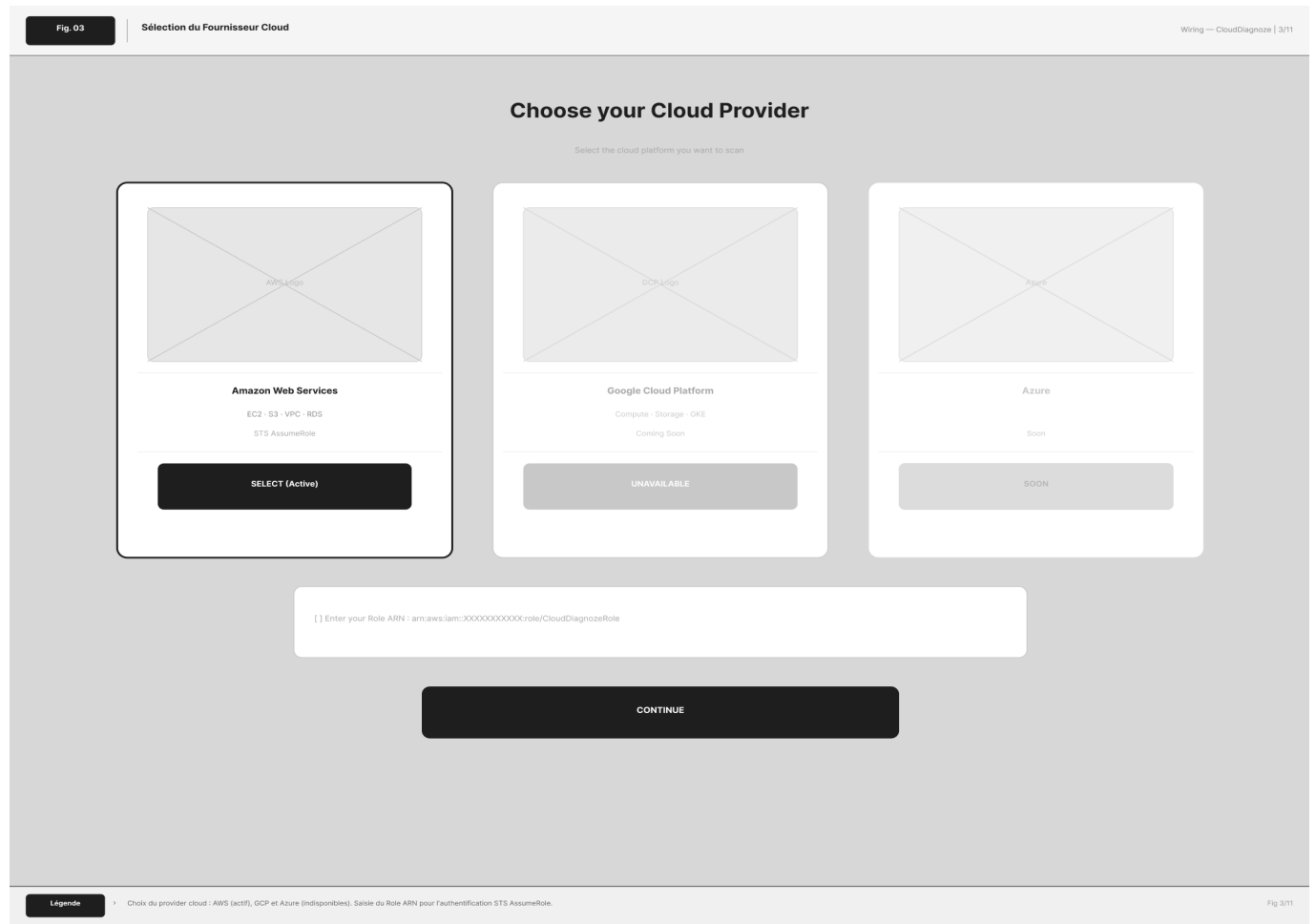
L'interface de CloudDiagnoze est une Single Page Application (SPA) développée en Vanilla JS. La conception visuelle a suivi un processus itératif classique :

Zoning

Le zoning a permis de définir les macro-zones de l'application sans se soucier du design graphique. Pour la page principale (Dashboard), l'interface est divisée en trois zones clés :

1. **Menu latéral (Sidebar) : Navigation entre les services (EC2, S3), l'historique des scans et les paramètres.**
2. **Barre supérieure (Topbar) : Affichage du profil utilisateur, du statut global et du bouton "Lancer un scan".**
3. **Contenu principal : Affichage dynamique des cartes de statistiques et des tableaux de ressources.**

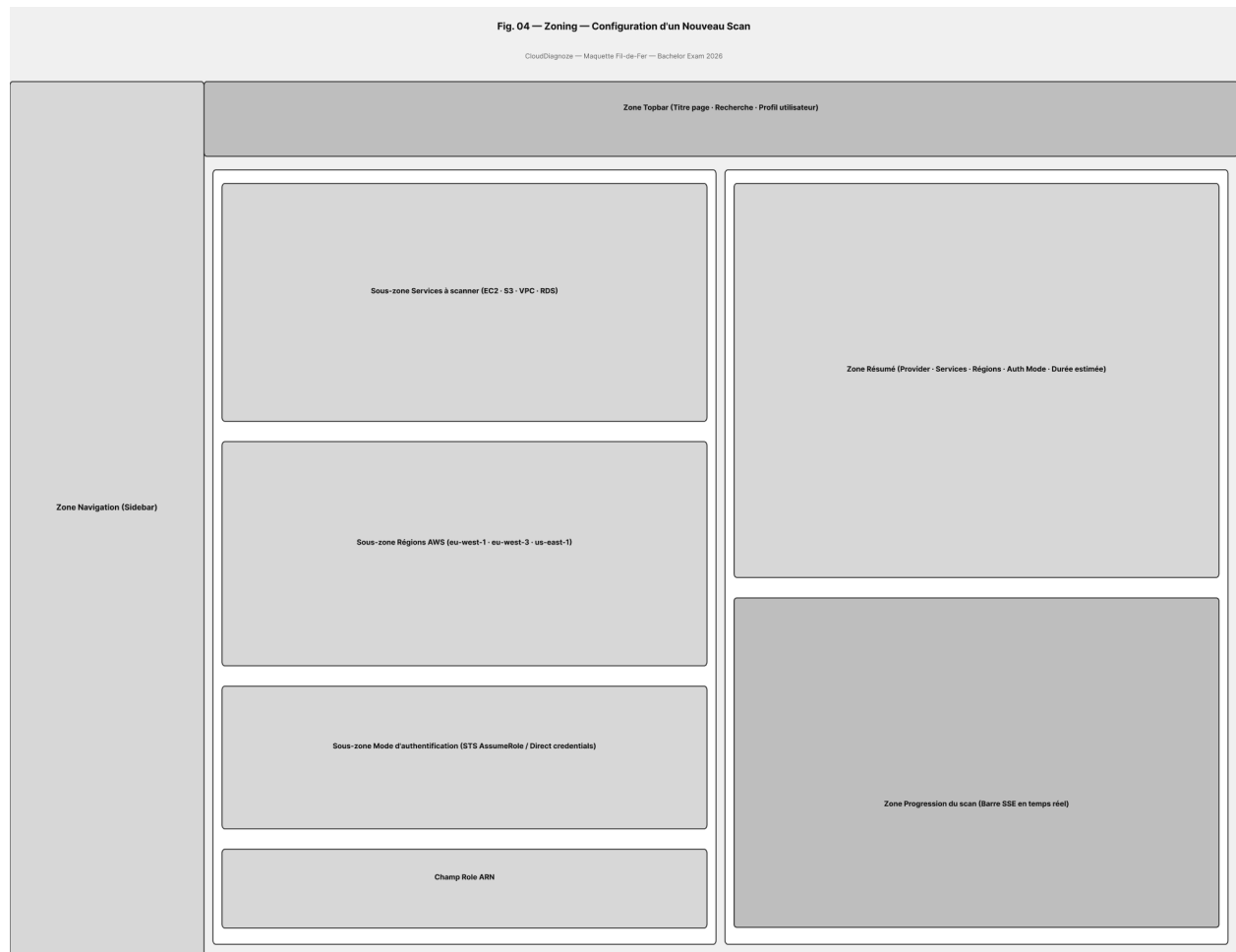




Wireframe

Les wireframes détaillent le placement des éléments (boutons, filtres, colonnes de tableaux) pour chaque vue métier. Par exemple, le wireframe de la configuration de scan

([config-scan-new.html](#)) illustre les étapes de sélection :

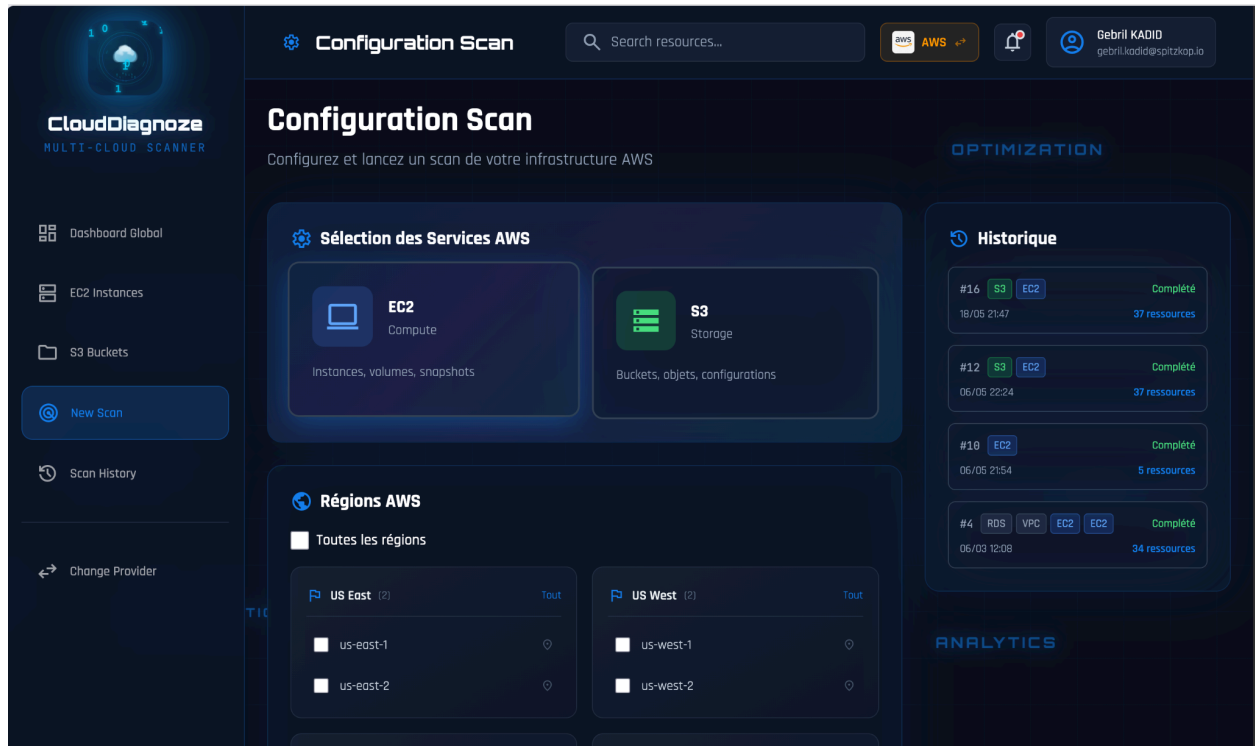


- Choix du fournisseur (AWS).
- Sélection des services à auditer.
- Saisie du Role ARN et des régions.

Maquette

Les maquettes finales intègrent la charte graphique (thème sombre, couleurs primaire **#137FEC** et accent **#06B6D4**, typographies *Rajdhani* et *Orbitron*). Elles représentent le rendu final des pages, incluant les animations CSS et les états interactifs.

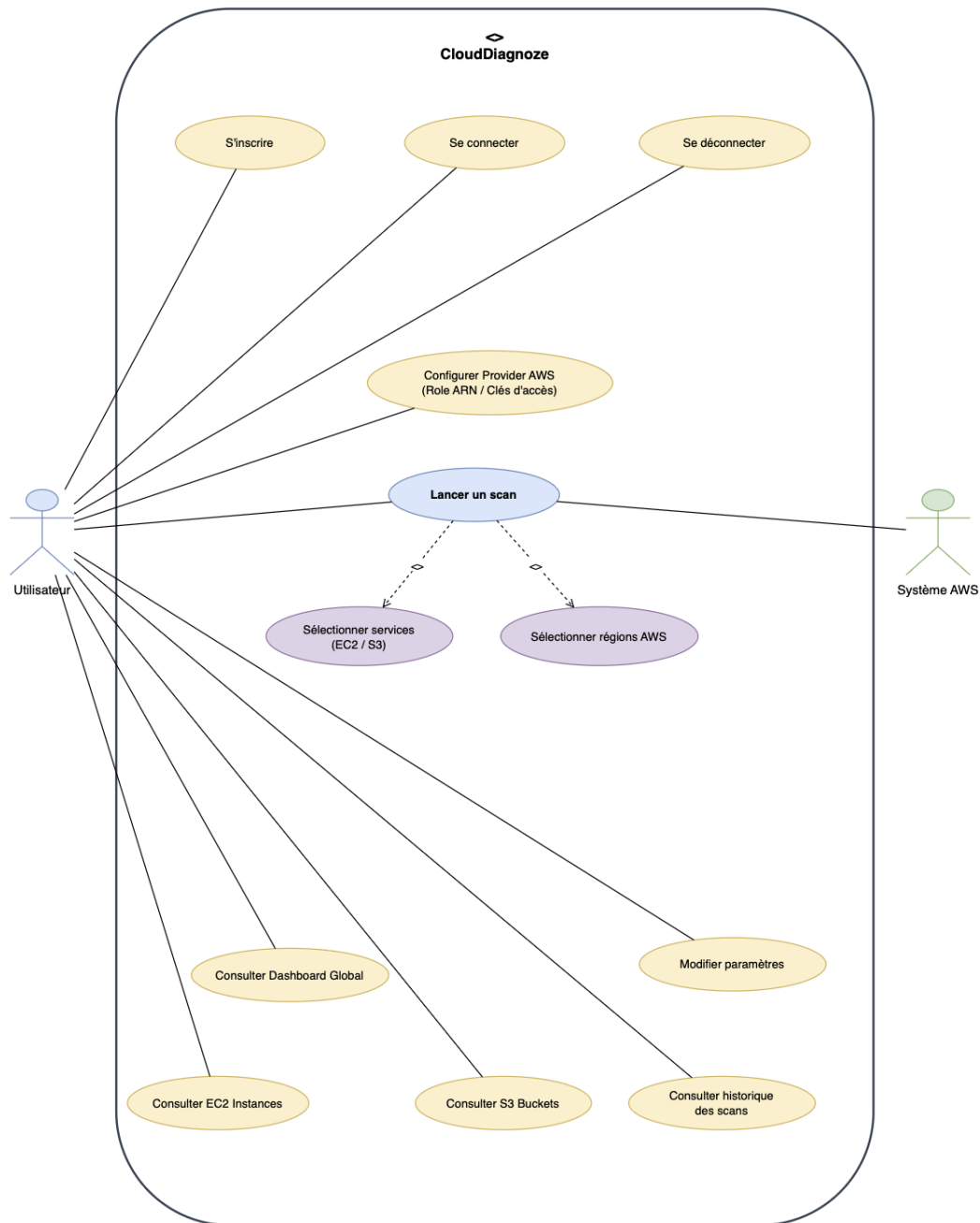




4.2. Back-end

La conception du Back-end repose sur une architecture robuste en Python/FastAPI, communiquant avec une base de données relationnelle MariaDB. **Diagramme UML Cas d'Utilisation**

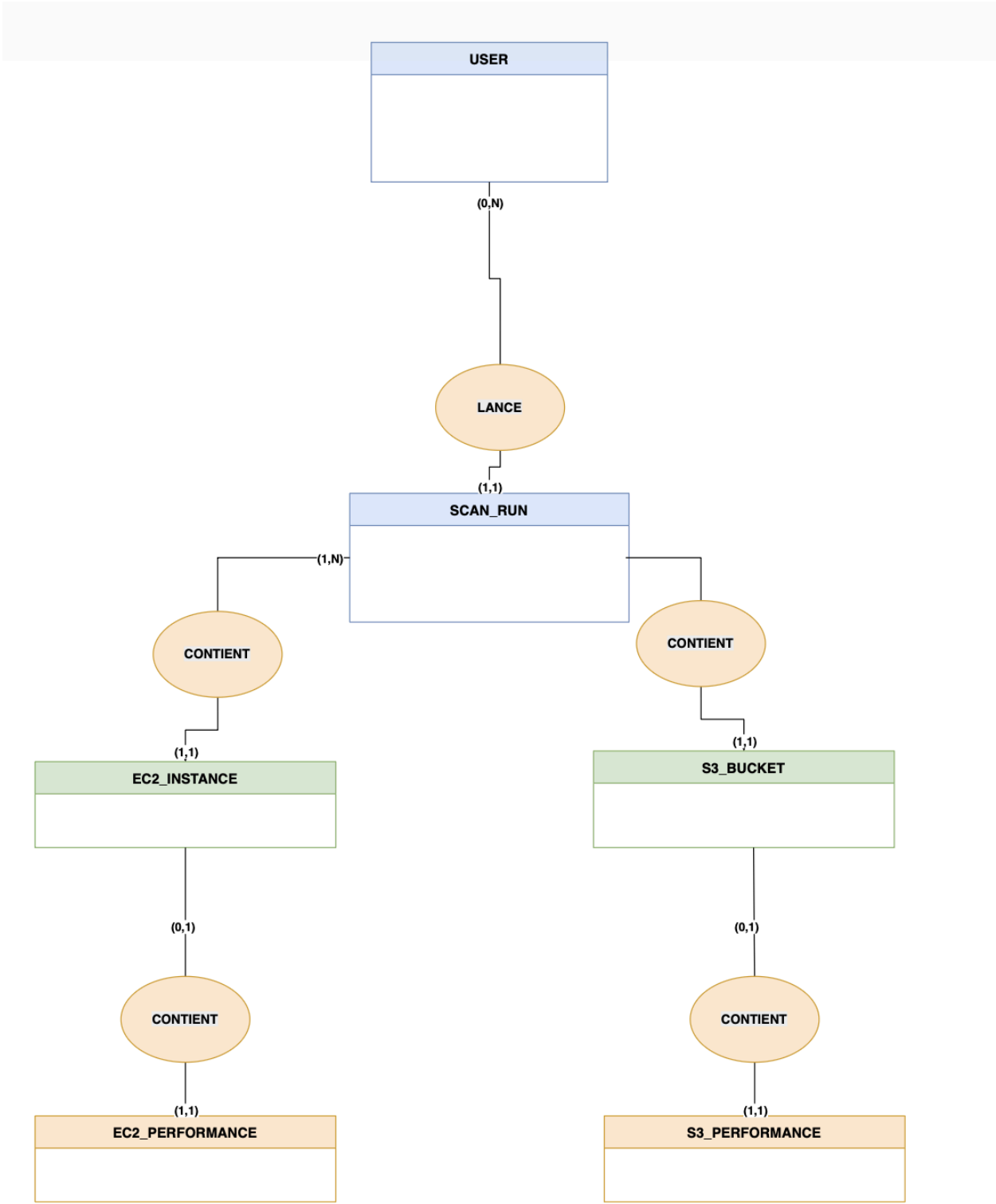
Ce diagramme met en évidence les interactions de l'utilisateur avec le système:



- **Utilisateur (Admin/DevOps)** : S'inscrire/Se connecter , configurer ses accès cloud via STS AssumeRole , lancer un scan d'infrastructure , visualiser les tableaux de bord , exporter les résultats en JSON.
- **Système AWS (Acteur secondaire)** : Fournit les métadonnées et métriques via l'API Boto3.

Modèle Conceptuel de Données (MCD)

Le MCD illustre les entités conceptuelles du domaine (DDD - Domain-Driven Design). Le cœur du modèle est l'entité User, liée à de multiples Scan Run (sessions de scan). Chaque Scan Run est lui-même lié aux ressources découvertes : EC2 Instance et S3 Bucket Instance. Chaque ressource possède une entité de performance associée (ex: EC2 Performance pour les métriques CPU/réseau, S3 Performance pour les métriques de stockage)



5. Mise en œuvre de la Sécurité

La gestion de la sécurité a été pensée dès la conception (Security by Design) pour protéger à la fois les données des utilisateurs, l'infrastructure de l'application et les environnements Cloud audités.

5.1. Authentification et Autorisation

L'accès à la plateforme est strictement contrôlé pour garantir l'isolation des données entre les différents utilisateurs.

- **Hachage des mots de passe** : Les mots de passe des utilisateurs ne sont jamais stockés en clair. Ils sont hachés et salés en utilisant l'algorithme `bcrypt`. Une politique de mot de passe fort est appliquée (minimum 8 caractères, majuscule, chiffre).
- **Jetons JWT (JSON Web Tokens)** : L'authentification repose sur des tokens JWT signés avec l'algorithme `HS256` et une clé secrète forte. Ce token est stocké côté client dans un cookie `httpOnly` sécurisé et transmis automatiquement dans les en-têtes HTTP de chaque requête API protégée.
- **Isolation et cycle de vie des données** : Chaque requête d'accès aux ressources est filtrée au niveau de la base de données par l'`user_id`. De plus, la suppression d'un compte utilisateur entraîne une suppression en cascade (`CASCADE DELETE`) de toutes ses sessions de scan et métriques associées.

```
# =====  
# FONCTIONS DE HASHING DE MOT DE PASSE  
# =====  
  
def hash_password(password: str) -> str:  
    """  
    Hash un mot de passe en utilisant bcrypt.  
  
    Args:  
        password: Mot de passe en clair  
  
    Returns:  
        Mot de passe hashé  
  
    Example:  
        >>> hashed = hash_password("MyPassword123")  
        >>> print(hashed)  
        $2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/LewY5GyYqVr/1jrYK  
    """  
    return pwd_context.hash(password)
```

```

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    """
    Crée un token JWT pour l'authentification.
    """
    Args:
        data: Données à encoder dans le token (ex: {"sub": "user@example.com"})
        expires_delta: Durée de validité du token (optionnel)
    Returns:
        Token JWT encodé
    """
    Example:
    >>> token = create_access_token({"sub": "john@acme.com", "user_id": 1})
    >>> print(token)
    eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

    to_encode = data.copy()

    # Définir la date d'expiration
    if expires_delta:
        expire = datetime.now(timezone.utc) + expires_delta
    else:
        expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    # Encoder le token
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

    logger.info(f" Token JWT créé (expire: {expire})")

    return encoded_jwt

```

5.2. Gestion des accès Cloud (AWS STS AssumeRole)

C'est le mécanisme de sécurité principal de l'application concernant l'interaction avec le Cloud cible.

- **Zéro clé statique exposée** : Contrairement aux pratiques risquées consistant à demander les **Access Key ID** et **Secret Access Key** du client, CloudDiagnoze utilise le service **AWS STS (Security Token Service)**.
- **Principe du AssumeRole** : L'utilisateur fournit uniquement le nom de ressource Amazon (Role ARN) d'un rôle IAM préalablement configuré sur son compte. Le moteur de scan (via Boto3) assume ce rôle pour obtenir des identifiants temporaires de courte durée.
- **Moindre privilège** : La portée de ces accès temporaires est strictement limitée aux permissions de lecture (**ReadOnlyAccess**) sur les services audités (**EC2, S3**).

```

def get_assumed_session(role_arn: str):
    """Pour facilit   le client ont cr     l'assume role a sa place"""
    """Chose a d  terminer en fonction de la technicit   du client"""
    logger.info(" D  marrage AssumeRole avec ARN : {}", role_arn)
    try:
        sts = boto3.client(
            "sts",
            aws_access_key_id=os.getenv("AWS_ACCESS_KEY_ID"),
            aws_secret_access_key=os.getenv("AWS_SECRET_ACCESS_KEY"),
            region_name=os.getenv("AWS_DEFAULT_REGION", "eu-west-1")
        )

        response = sts.assume_role(
            RoleArn=role_arn,
            RoleSessionName="CloudDiagnozeSession"
        )

        logger.success(" AssumeRole r  ussi. Expire    : {}", response['Credentials']['Expiration'])

        credentials = response['Credentials']
        session = boto3.Session(
            aws_access_key_id=credentials['AccessKeyId'],
            aws_secret_access_key=credentials['SecretAccessKey'],
            aws_session_token=credentials['SessionToken'],
            region_name=os.getenv("AWS_DEFAULT_REGION", "eu-west-1")
        )
        return session

```

5.3. Protection de l'infrastructure et du R  seau

L'infrastructure de d  ploiement de CloudDiagnoze int  gre plusieurs couches de protection r  seau.

- **Proxy Inverse Nginx** : Le conteneur Nginx agit comme point d'entr  e unique. Il masque l'architecture interne et applique des en-t  tes de s  curit   HTTP stricts tels que **X-Frame-Options**, **X-Content-Type-Options** et **X-XSS-Protection**.
- **Rate Limiting (Limitation de d  bit)** : Pour contrer les attaques par force brute ou les d  nis de service (DDoS), la biblioth  que **SlowAPI** est int  gr  e au backend FastAPI. Des limites strictes sont appliqu  es sur les routes sensibles : 5 requ  tes/minute pour l'inscription, 10 requ  tes/minute pour la connexion et 3 requ  tes/minute pour la r  initialisation de mot de passe.
- **Isolation Docker** : La base de donn  es MariaDB n'est expos  e sur aucun port public. Elle n'est accessible que via le r  seau virtuel interne Docker (**clouddiagnoze-network**) par le conteneur Backend.
- **Gestion des Secrets** : L'ensemble des variables d'environnement (mots de passe BDD, cl   secr  te JWT, configuration CORS) est g  r   via un fichier **.env** non commit   sur le d  p  t de code, g  n  r   de mani  re s  curis  e lors de l'ex  cution du script **setup.py**.

```

5 # =====
4 # R  SEAU INTERNE
3 # =====
2 networks:
1 | clouddiagnoze-network:
125 | driver: bridge

```

```

docker-compose.yml | limiter.py 2
5 from slowapi import Limiter • Import "slow
4 from slowapi.util import get_remote_address
3
2 # Initialisation du Limiter
1 # key_func=get_remote_address : Limite basée s
6 limiter = Limiter(key_func=get_remote_address)

```

6. Qualité Logicielle et Tests

Pour garantir la fiabilité de **CloudDiagnoze** et s'assurer que les évolutions du code n'introduisent pas de régressions, une stratégie de tests automatisés rigoureuse a été mise en place, couplée à un pipeline d'intégration et de déploiement continu (CI/CD).

6.1. Stratégie de tests unitaires (Mocking via Moto)

Les tests unitaires se concentrent sur la validation de la logique métier isolée, en particulier les scanners AWS (test_ec2_scanner.py, test_s3_scanner.py,).

- **Framework** : Utilisation de pytest couplé à pytest-asyncio pour supporter l'exécution asynchrone du moteur de scan.
- **Mocking AWS avec Moto** : Pour tester les interactions avec l'API AWS sans nécessiter de véritable compte Cloud ni générer de coûts, la bibliothèque moto a été intégrée. Elle permet de simuler (mock) l'infrastructure AWS (création de fausses instances, buckets, etc. via des fixtures) de manière transparente pour le SDK Boto3.
- **Couverture de code** : Un rapport de couverture de code peut être généré via la commande `pytest --cov=api tests/` pour s'assurer que les cas d'usage principaux sont bien testés.

```

pytest tests/aws_tests/ -v
tests/aws_tests/test_ec2_scanner.py::TestEC2ScannerErrors::test_scan_with_mixed_valid_invalid_regions PASSED [ 18%]
tests/aws_tests/test_ec2_scanner.py::TestEC2ScannerMetrics::test_scan_instance_with_cpu_metrics PASSED [ 21%]
tests/aws_tests/test_ec2_scanner.py::TestEC2ScannerMetrics::test_scan_instance_with_network_metrics PASSED [ 25%]
tests/aws_tests/test_ec2_scanner.py::TestEC2ScannerMetrics::test_scan_instance_without_metrics PASSED [ 28%]
tests/aws_tests/test_ec2_scanner.py::TestEC2ScannerEdgeCases::test_scan_stopped_instance PASSED [ 31%]
tests/aws_tests/test_ec2_scanner.py::TestEC2ScannerEdgeCases::test_scan_instance_with_tags PASSED [ 34%]
tests/aws_tests/test_ec2_scanner.py::TestEC2ScannerEdgeCases::test_scan_instance_different_types PASSED [ 37%]
tests/aws_tests/test_ec2_scanner.py::TestEC2ScannerIntegration::test_scan_data_format_validation PASSED [ 40%]
tests/aws_tests/test_ec2_scanner.py::TestEC2ScannerIntegration::test_scan_arn_format PASSED [ 43%]
tests/aws_tests/test_ec2_scanner.py::TestEC2ScannerIntegration::test_scan_with_no_regions_specified PASSED [ 46%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerBasic::test_scan_no_buckets PASSED [ 50%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerBasic::test_scan_single_bucket PASSED [ 53%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerBasic::test_scan_multiple_buckets PASSED [ 56%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerBasic::test_scan_buckets_different_regions PASSED [ 59%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerConfigurations::test_scan_bucket_with_encryption PASSED [ 62%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerConfigurations::test_scan_bucket_with_versioning PASSED [ 65%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerConfigurations::test_scan_bucket_with_public_access_block PASSED [ 68%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerConfigurations::test_scan_bucket_with_lifecycle PASSED [ 71%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerConfigurations::test_scan_bucket_with_website PASSED [ 75%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerMetrics::test_scan_bucket_with_request_metrics PASSED [ 78%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerMetrics::test_scan_bucket_with_error_metrics PASSED [ 81%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerMetrics::test_scan_bucket_without_metrics PASSED [ 84%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerEdgeCases::test_scan_bucket_with_all_features PASSED [ 87%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerEdgeCases::test_scan_bucket_with_no_features PASSED [ 90%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerIntegration::test_scan_data_format_validation PASSED [ 93%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerIntegration::test_scan_arn_format PASSED [ 96%]
tests/aws_tests/test_s3_scanner.py::TestS3ScannerIntegration::test_scan_with_no_regions_specified PASSED [100%]

```

6.2. Tests d'intégration et couverture

Les tests d'intégration valident le bon fonctionnement de l'application dans son ensemble, de la requête HTTP jusqu'à la persistance en base de données.

- **Prérequis technique** : Contrairement aux tests unitaires, ces tests (tests/integration/) nécessitent qu'une base de données MariaDB réelle soit en cours d'exécution.
- **Périmètre testé** :
 - Le flux complet d'authentification (`test_auth_endpoints.py`).
 - Le cycle de vie complet d'un scan (`test_scan_endpoints.py`).
 - Le bon fonctionnement du streaming SSE (Server-Sent Events) pour la remontée de la progression en temps réel (`test_events_endpoints.py`).

6.3. Pipeline CI/CD (GitHub Actions)

L'automatisation du cycle de vie du code est gérée par un pipeline CI/CD moderne utilisant GitHub Actions, hébergé dans le dépôt sous `.github/workflows/ci.yml`.

- **Exécution locale et distante** : Le pipeline s'exécute automatiquement sur chaque ``push`` ou ``pull_request`` ciblant la branche ``master``, directement dans l'environnement GitHub (ubuntu-latest).
- **Étapes du pipeline** :
 1. Exécution des tests unitaires avec le mock Moto (EC2, S3).
 2. Exécution des tests d'intégration (qui démarre une instance MariaDB éphémère).
 3. Construction (Build) de l'image Docker du backend et push (déploiement).
 4. Construction (Build) de l'image Docker du frontend et push.
 5. Validation finale de la configuration `docker-compose`.

The screenshot shows a GitHub Actions workflow run for the pull request "Merge pull request #9 from djibs92/fix/clean-up-add-feature #4". The workflow is titled "Docker Compose Build" and has a status of "succeeded 19 hours ago in 1m 8s". The workflow steps are listed in a table:

Step	Duration
Set up job	2s
Checkout	2s
Set up Docker Buildx	8s
Create dummy .env for build	0s
Build Docker images	53s
Post Set up Docker Buildx	1s
Post Checkout	0s
Complete job	0s

7. Manuel de déploiement

Le déploiement de **CloudDiagnoze** a été conçu pour être le plus fluide et automatisé possible, garantissant une reproductibilité parfaite entre les environnements de développement et de production grâce à la conteneurisation.

7.1. Configuration de l'environnement (.env)

La gestion des secrets et des paramètres d'environnement est centralisée dans un fichier `.env`, généré à partir d'un template `.env.example`. Ce fichier permet de configurer dynamiquement les conteneurs sans exposer de données sensibles dans le code source.

Les variables principales couvrent trois domaines :

- **Base de données (MariaDB)** : `DB_ROOT_PASSWORD`, `DB_NAME` (par défaut `clouddiagnoze`), `DB_USER` et `DB_PASSWORD`.
- **Sécurité JWT** : `SECRET_KEY` (clé cryptographique forte), `JWT_ALGORITHM` (configuré sur HS256), et `ACCESS_TOKEN_EXPIRE_MINUTES`.
- **Cloud Provider (AWS)** : `AWS_DEFAULT_REGION` et, si nécessaire pour des tests locaux hors STS, les clés d'accès (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`).

7.2. Procédure d'installation (Setup script)

Pour simplifier la configuration initiale, un script Python interactif (`setup.py`) a été développé pour guider l'administrateur pas à pas.

Le script exécute les 5 étapes suivantes :

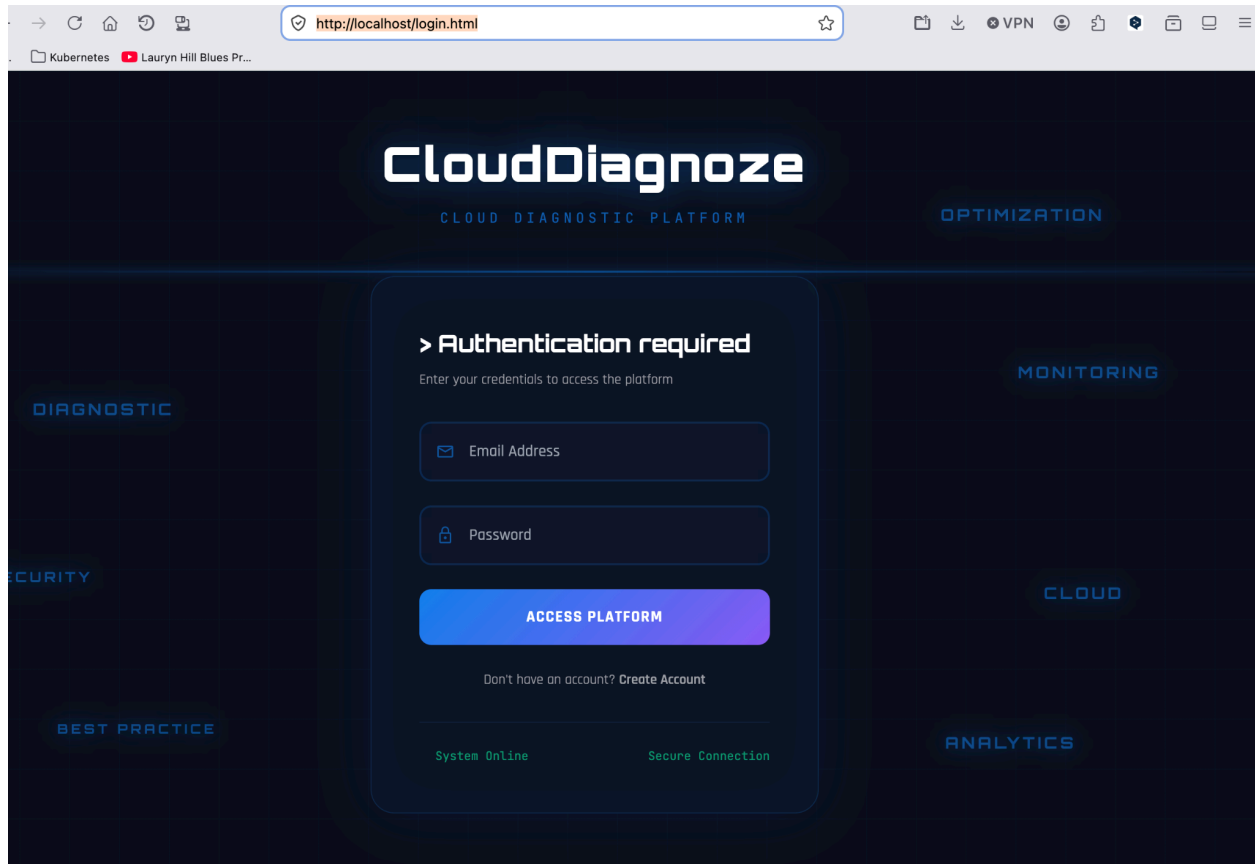
1. **Étape 0 - Prérequis** : Vérification de la présence de Docker et Docker Compose sur le système hôte.
2. **Étape 1 - Base de données** : Saisie interactive des identifiants et mots de passe pour MariaDB.
3. **Étape 2 - Sécurité** : Génération automatique d'une clé JWT hautement sécurisée (via `secrets.token_urlsafe(32)`).
4. **Étape 3 & 4 - Configuration Cloud et Génération** : Saisie de la région AWS par défaut et création automatique du fichier `.env` final.
5. **Étape 5 - Lancement** : Proposition de démarrage immédiat de l'infrastructure via la commande `docker compose up -d`.

Une fois le script validé, l'orchestrateur Docker Compose lève l'architecture complète composée de trois services interdépendants isolés sur un réseau interne (`clouddiagnoze-network`):

- **mariadb (Port 3306)** : Le serveur de base de données, qui s'initialise automatiquement avec le schéma SQL (`init_db.sql`).
- **backend (Port 8000)** : L'API FastAPI, qui attend que la base de données soit déclarée healthy avant de démarrer.
- **frontend (Port 80)** : Le serveur Web et Reverse Proxy Nginx, qui sert l'application SPA et redirige les requêtes API vers le backend de manière transparente.

```
docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
PORTS					
clouddiagnoze-backend	bachelor_exam-backend	"uvicorn main:app --..."	backend	44 hours ago	Up 35 seconds (healthy)
0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp					
clouddiagnoze-db	mariadb:11.2	"docker-entrypoint.s..."	mariadb	44 hours ago	Up 35 seconds (healthy)
0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp					
clouddiagnoze-frontend	bachelor_exam-frontend	"/docker-entrypoint...."	frontend	44 hours ago	Up 35 seconds (healthy)
0.0.0.0:80->80/tcp, [::]:80->80/tcp					



Conclusion et Perspectives

Bilan du projet

Le développement de CloudDiagnoze a permis de concevoir et de déployer avec succès une plateforme SaaS fonctionnelle, répondant aux exigences strictes de l'audit d'infrastructure Cloud. L'objectif principal de fournir une visibilité claire, rapide et sécurisée sur les ressources critiques (EC2, S3) d'un compte AWS a été atteint.

La réussite de ce projet Bachelor repose sur des choix architecturaux forts et modernes :

- L'utilisation d'une authentification par délégation de rôle (**AWS STS AssumeRole**), garantissant qu'aucune clé statique compromettante n'est stockée par l'application.
- Une conception logicielle modulaire basée sur des *Design Patterns* (Factory, Strategy) séparant distinctement le moteur de scan de la persistance des données.
- Une infrastructure robuste, asynchrone et entièrement conteneurisée (FastAPI, MariaDB, Docker) accompagnée d'un pipeline CI/CD moderne (GitHub Actions) assurant la qualité continue du code.

CloudDiagnoze démontre ainsi des pratiques d'ingénierie logicielle de niveau entreprise, fournissant une base solide pour des diagnostics approfondis.

Perspectives et évolutions futures

Bien que le produit actuel constitue un MVP (Minimum Viable Product) très avancé, l'architecture a été pensée pour être hautement extensible. Plusieurs axes d'amélioration ont été identifiés pour les itérations futures :

1. **Ouverture Multi-Cloud** : L'architecture actuelle par "fournisseur" permet d'intégrer techniquement de nouveaux scanners pour Microsoft Azure et Google Cloud Platform (GCP) sans refondre le moteur principal.
2. **Méthodologie 2CBP Avancée** : Approfondir l'analyse selon les piliers *Cost* (intégration d'AWS Cost Explorer), *Compliance* (vérifications des normes CIS, PCI-DSS) et *Best Practices* (recommandations actionnables).
3. **Intelligence Artificielle et Prédicatif** : Intégrer des modèles de Machine Learning pour réaliser des analyses prédictives, anticiper les tendances d'utilisation des ressources et prévoir précisément les coûts futurs.
4. **Notifications en temps réel** : Faire évoluer le système de *polling* actuel vers une communication bidirectionnelle via WebSockets pour des alertes instantanées.
5. **Remédiation automatique** : Passer d'un outil d'audit passif à un outil proactif capable d'appliquer automatiquement des corrections (ex: fermer un port de sécurité ouvert par erreur ou éteindre une instance inutilisée).